AI 전문가 양성을 위한「 KB AI 엘리트 교육」과정

TxLens

마이데이터 거래내역 정규화 · 분류 + 패턴 탐지 에이전트

> " 가맹점명을 못 묶으면 구독도 못 찾는다 "



6 주차 최종 발표 · 2026-10-17 (토)

한눈에 보기

TxLens: 거래내역 정규화 · 분류 + 패턴 탐지

- 제각각인 가맹점명 정리 + 카테고리 분류 → 그 위에서 구독 · 이상지출 · 이상거래 탐지
- LangChain 1.0 `create_agent` · LangGraph · Pydantic v2 · Streamlit
- 1 부 ·2 부 · 화면 4 개 · 사람 교정까지 동작 확인 (테스트 384 개)
- 정확도 : 룰만 93% / LLM 만 86% / 하이브리드 99% → 예상과 순서가 달랐음

해결하려는 현업 문제

아래 여섯 줄 , 전부 같은 가맹점

```
  ( 주 ) 스타벅스커피코리아 1234 호점
  STARBUCKS  1234
스타벅스 강남역점
  NAVERPAY* 스타벅스
  ( 주 ) 에스씨케이컴퍼니      <-  법인명만 온다 .  브랜드를 알 수 없다
  ( 주 ) 케이지이니시스        <-  PG  경유 .  실제 가맹점이 사라졌다
```

	내용
누가	지출분석 화면을 보는 고객 · 룰 사전을 관리하는 담당자
언제	거래내역이 들어올 때마다 (신규 브랜드 · 지점은 매일 생김)
왜 생기나	표준 API 가 필드 구조만 정하고 가맹점명은 사실상 자유 텍스트
결과 1	카테고리 오분류 → 지출분석 화면을 못 믿게 되고 안 쓰게 됨
결과 2	' 기타 ' 증가 → 분석 가치 하락 , CS 문의 증가
결과 3	사전 수작업 관리 → 새 표기가 계속 들어와 끝이 안 남

왜 룰만으로는 안 되는가

하나 . 롱테일이라 , 같은 룰도 세는 기준에 따라 93% vs 42%

룰 사전 v1 (head 29 개) 로 측정	거래 건수 기준	정규화 키 기준
L2 정확도	93.0%	42.0%
미매칭	7.7%	58.7% (키 264 개 중 155 개)

둘 . 표기 편차가 카드사 · 페이사별로 뭉쳐 있음 (실무 확인)

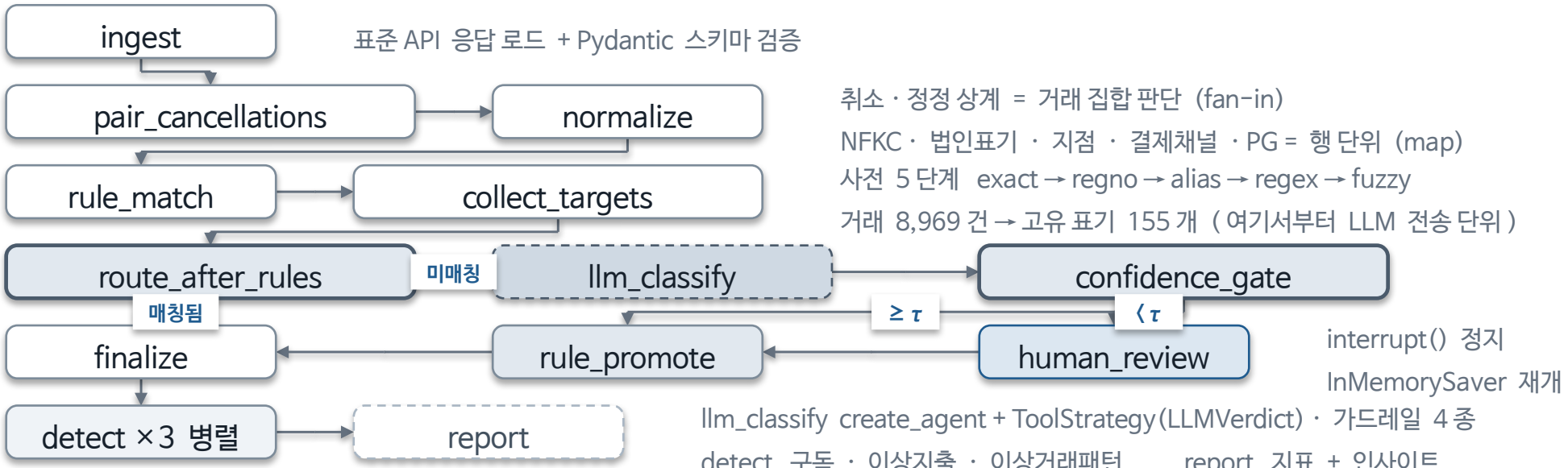
기관	비중	regno 채움	지배적 표기	난이도
A 카드	30%	95%	브랜드명 · 지점명	쉬움
B 카드	25%	50%	법인명 (브랜드 불명)	어려움
C 카드	20%	20%	PG 노출 25%	매우 어려움
D 페이	15%	35%	간편결제 프리픽스 70%	중간
E 카드	10%	10%	전부 섞임 , 오타 10%	매우 어려움

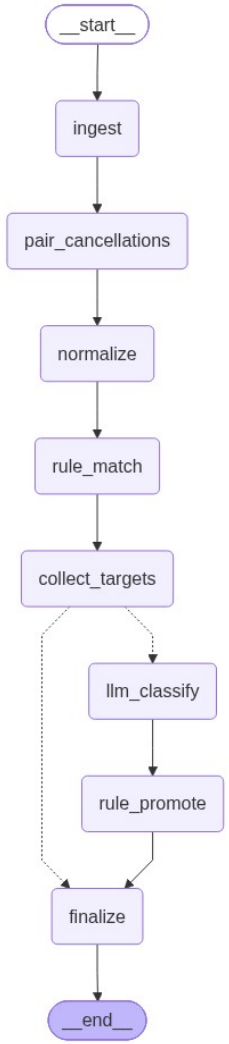
무엇을 하는 에이전트인가

1 부가 이름을 묶어야 2 부가 찾을 수 있음

	1 부 : 정규화 · 분류	2 부 : 패턴 탐지
입력	표준 API 형태 거래내역	1 부의 출력
방법	룰 (head) + LLM (tail) + 사람 교정 → 사전에 반영	통계 규칙 (LLM 안 씀)
출력	`merchant_canonical` · L1/L2 카테고리 · 신뢰도	구독 목록 · 이상지출 알림 · 이상거래 알림
지표	정확도 · 미분류율 · LLM 호출 감소	precision / recall

기능 흐름도





실제 그래프 (C)

- 설계 노드 13 개 중 LLM 이 들어가는 건 `llm_classify` 하나. 나머지는 테스트로 고정
- 조건부 분기 2 개 (`route_after_rules`·`confidence_gate`), 사람 개입 1 개, 사전으로 돌아오는 루프 1 개
- 지금 실제로 컴파일되는 노드는 8 개 (`rule_promote` 포함)
- `build_graph(review=True)` 로 켜면 승격 자리에 사람 검토 서브그래프가 들어감
- 탐지 3 종은 그래프 노드가 아닌 일반 함수 (`finalize` 결과를 받음)
  - 오른쪽 그림 : `get_graph()` 출력 그대로
- **A/B/C 세 구성 모두 이 그래프 하나로 돎.** 바꾸는 건 사전과 분류기뿐

State 설계 : 스키마는 TypedDict, 필드는 Pydantic

	선택	이유
상태 스키마 `TxState`	`TypedDict (total=False)`	노드는 바뀐 키만 반환 . Pydantic 을 스키마로 쓰면 갱신 때마다 전체 재검증 (8,969 건 × 노드 8 개)
필드 타입	Pydantic `BaseModel`	단계 간 계약 : `RawTx` → `NormalizedTx` → `ClassifiedTx`. 단계마다 보장하는 게 타입에 드러남

reducer 는 필드 성질에 맞춰 선택

성질	필드	reducer
다음 단계가 대체한다	`raw·pairings·normalized·matched·confirmed·metrics`	없음 (교체)
여러 노드 · 라운드가 덧붙인다	`review_queue·human_corrections·rule_proposals·llm_failed`	`operator.add`
표기 단위 결과를 합친다	`llm_results`	`merge_verdicts` (나중 값 우선)

- `Annotated[..., operator.add]` 는 편하지만 값이 계속 쌓이는 점 주의
- `llm_results` 키 = 정규화 표기 = 캐시 키
- `human_corrections` 는 쓰는 쪽과 읽는 쪽이 둘 다 생긴 뒤에 추가 (`insights` 가 아직 없는 이유)
- 검토 서브그래프는 **별도 스키마** (`ReviewState`). 거래 데이터를 안 들고 감

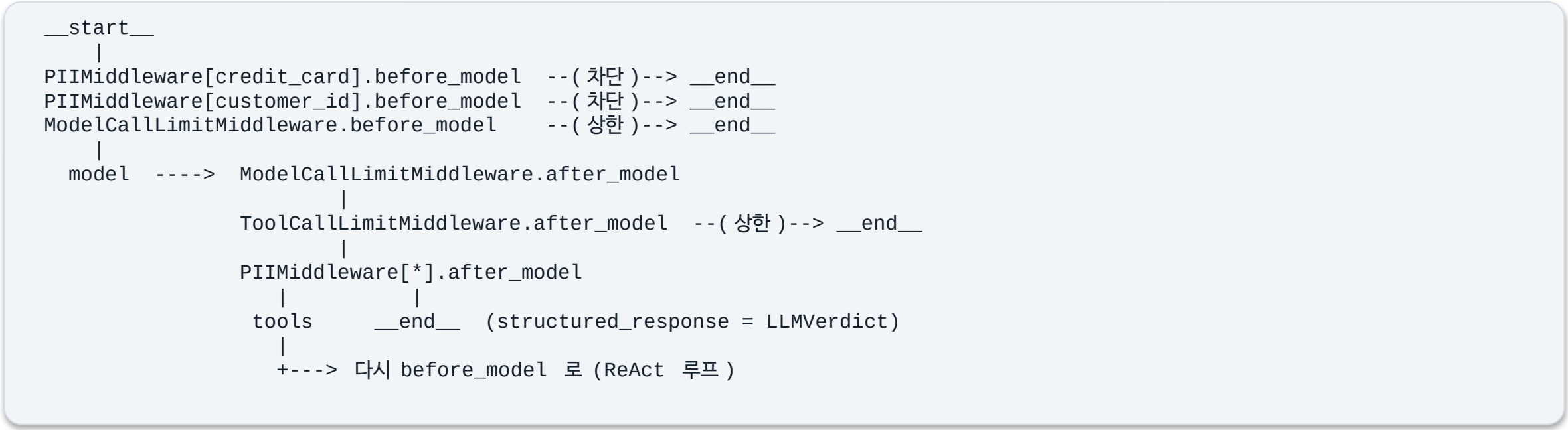
Node · Edge 설계 : 어디서 나눌까

분리한 노드	근거
`preprocess` → `pair_cancellations` + `normalize`	앞은 (기관, 고객, 승인번호)로 묶어 합계를 보는 fan-in, 뒤는 행 단위 map
`rule_match` → `rule_match` + `collect_targets`	처리 단위가 거래 → 고유 표기로 바뀌는 지점. 여기서 LLM 비용이 정해짐

Edge: 조건부 분기 2 개 + 일부러 안 만든 분기

- `route_after_rules`: 미매칭이 없으면 `llm_classify` 건너뜀
- `confidence_gate`: 신뢰도 τ 미만만 `human_review` 로. **볼 게 없으면 사람을 안 부름**
- (A) 는 `classifier=None` 이라 그 노드와 분기 자체를 안 만듦
- Send 로 표기별 병렬 처리는 안 함 (호출 수를 결정론으로 유지)

llm_classify 노드 안 : 가드레일도 노드



- 가드레일 4 종이 그래프 노드로 들어감 : `PIIMiddleware` 2 종 · `ModelCallLimit` · `ToolCallLimit`
- `model` ↔ `tools` 루프 . 표기 155 개에 모델 호출 580 회
- **개인정보 차단은 프롬프트가 아닌 미들웨어에서**
- 위 그림 : `agent.get_graph()` 출력 그대로

Agent 입출력 : 도구 3 개 + structured output

입력 : 사전을 프롬프트에 다 넣지 않고 도구로 조회

도구	하는 일	왜 도구인가
`lookup_alias`	별칭 사전 유사 조회 (`token_sort_ratio` ≥ 55)	사전이 커져도 프롬프트는 그대로
`search_taxonomy`	택소노미에서 후보 카테고리 검색	호출하면 더 정확 (을지로호프)
`get_brand_hint`	전처리 산출물 조회	이미 뽑은 값은 다시 추론 안 시킴

출력 : response_format=ToolStrategy(LLMVerdict) 한 줄

막으려는 값	막는 장치
택소노미에 없는 카테고리	`CategoryL1·CategoryL2` Enum (자유 문자열 불가)
`l1=food` + `l2=cafe.coffee`	`model_validator` (실제로 나왔던 조합)
`confidence=1.7`	`Field(ge=0, le=1)`
근거 없는 판정	`reasoning·evidence` 필수 (검토자가 근거를 봐야 함)

도구를 밖으로 : 사람과 에이전트가 같은 도구를 씀

내보낸 것 : python -m txlens.mcp_server (stdio, 외부 네트워크 없음)

종류	무엇	쓰는 쪽
에이전트 도구 3 종	`lookup_alias` · `search_taxonomy` · `get_brand_hint`	에이전트와 사람
사람용 도구 2 종	`classify_merchant` · `dictionary_status`	사람만
리소스 2 종	`txlens://taxonomy` · `txlens://dictionary`	붙여 읽는 자료

도구는 한 벌만 유지

- 기존 `make_tools()` 를 감싸기만 함 . 도구 설명도 같은 걸 쓰고 , 둘이 같은지 테스트로 확인
- 실행 설정은 `.mcp.json` 한 곳 . Claude Code 와 우리 클라이언트가 같은 서버에 붙음

기본 실행 경로는 그대로 (전달 방식 때문에 수치가 바뀌면 안 되므로)

- `llm_classify` 는 in-process 그대로 . MCP 경유로 바꾸면 그래프를 async 로 바꾸고 A/B/C 를 다시 재야 함
- 대신 `mcp_server/client.py` 로 **두 경로의 답이 같은지** 확인만 해 둠

기술 스택 : 배운 걸 어디에 썼나

배운 기술 (LangChain 1.0 이후)	TxLens 에서 쓰는 자리	지금
`create_agent`	`llm_classify` 노드 안 (tail 분류만)	실측
`ToolStrategy(LLMVerdict)`	분류 출력 스키마	실측
`PIIMiddleware(strategy="block")` ×2	모델 호출 직전 차단	테스트
`ToolCallLimit·ModelCallLimit`	실행당 상한 `run_limit`	테스트
`AgentMiddleware.wrap_model_call`	호출 캐시 (히트면 모델 안 부름)	테스트
`InMemorySaver` / `InMemoryStore`	`human_review` 재개 / 룰 사전	실사용
`LLMToolEmulator` + 스크립트된 모델	API 키 없이 그래프 경로 · 툴 콜 순서 고정	384 tests
MCP `FastMCP` (stdio 서버)	에이전트 도구를 프로세스 밖으로	실사용
`langchain_mcp_adapters`	같은 서버에 우리 코드로도 붙어 답을 대조	테스트

세 가지 구성 비교 : 예상이 빗나간 곳

구성	거래 L2	키 L2	미매칭	확정 커버리지	LLM 키 / 모델 호출
(A) 룰만	93.0%	42.0%	7.7%	84.3%	0 / 0
(B) LLM 만 (사전 없음)	85.6%	66.7%	5.1%	86.8%	257 / 1,026
(C) 하이브리드	98.9%	86.7%	1.7%	90.3%	155 / 580

- 예상 : LLM 만 쓰는 B 가 제일 정확 → 실제로는 **룰만 쓴 A 보다 7%p 낮음**
- B 는 답을 많이 내지만 틀린 답도 자신 있게 냄 (쇼핑 500 건을 식비로 분류)
- 같은 모델 · 프롬프트라도 하이브리드 안에서는 LLM 정확도 88% → 97%
- PG 사 경유로 가맹점명이 지워진 8% 는 어떤 구성으로도 못 품

사전 승격 루프 : 자동은 별칭까지 , 새 브랜드는 사람이

라운드	사전 키	LLM 표기	모델 호출	거래 L2	키 L2	자동 승격
1 · v1	65	155	572	98.0%	87.9%	35 건
2 · v2	100	119	438	98.4%	90.5%	8 건
3 · v3	108	111	394	98.1%	91.7%	1 건
4 · v4	115	104	377	98.0%	90.2%	6 건
5 · v5	121	98	351	97.7%	90.2%	3 건

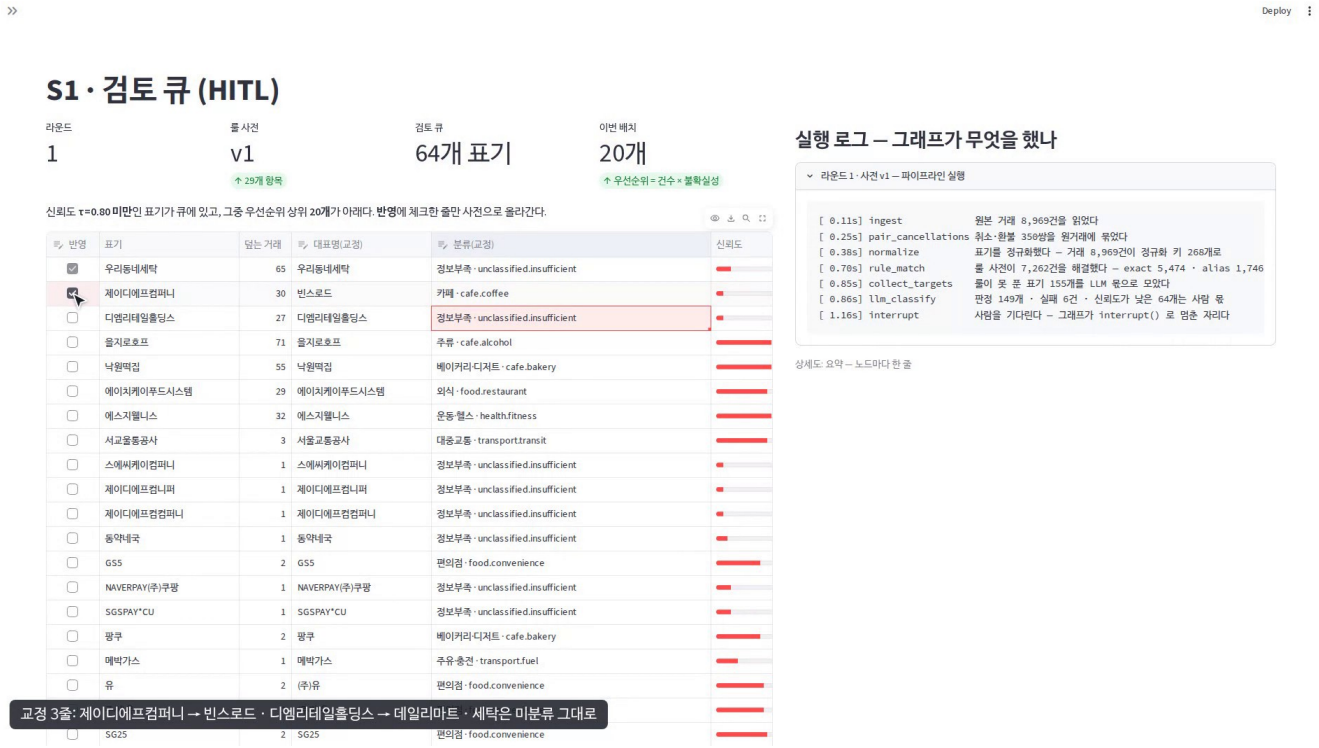
- 판정 근거별 정확도를 정답셋으로 측정 : 브랜드 인식 96% · 키워드 79% · 추측 9%
- 진짜 위험은 대표명 . 타벅스가 새 브랜드로 굳으면 스타벅스가 둘로 쪼개짐 → 자동 승격은 아는 브랜드의 별칭만
- **5 라운드 내내 수렴하지 않음** . 오타가 계속 새로 나옴 (GS2·GS5·GS52·GSS25…)
- 승격 판정은 거의 다 맞음 (1 라운드 40 건 중 39 건 · 4~5 라운드 13 건 전부). 호출 37% 감소 , 멈출 시점은 사람이 판단
- 검토 화면 (S1) 에서 세 줄 고치면 사전 29 → 32, 거래 122 건 바로 반영 , 다음 라운드 표기 155 → 115

2 부 탐지 : 분류가 좋아지면 탐지도 는다

분류 단계	미묶음	구독 P/R	이상지출 P/R	이상거래 P/R
정규화만 (사전 ·LLM 없음)	100%	95.3 / **56.2**	— / 0.0	96.4 / 79.4
(A) 룰만	15.7%	94.7 / 74.0	63.2 / 61.0	97.1 / 100.0
(C) 하이브리드	9.6%	94.7 / 74.0	62.5 / 67.8	97.1 / 100.0
정답 분류 (상한)	8.0%	95.7 / **90.4**	62.7 / 71.2	97.1 / 100.0

- 이름을 못 묶으면 구독은 절반 (56%) 만 잡힘 . 넷플릭스가 표기 3 개로 쪼개지는 식
- 이상지출은 0%, 이상거래는 79% 유지 . 카테고리에 기대는 정도 차이
- 남은 구독 손실 (74% → 90%) 은 대부분 **사전이 잘못 묶은 것** (쿠팡와우 → 쿠팡)
- 일부러 심은 오탐 함정 60 건 중 통과 0 건
- 고객당 알림 월 0.6 건 · 급증 정밀도 37% 는 보수적으로 잡힌 값

화면으로 확인한 것



S1 검토 큐 · 왼쪽이 사람이 고치는 자리 , 오른쪽이 8 노드 실행 로그

- 영상 네 구간 : S1 교정 (1:08) → S4 토글 (0:36) → S2 지표 (0:24) → 준비 상태 (0:08)
- 오른쪽 칸 로그가 그래프가 도는 증거 — 지연을 뺀 실제 노드 시간
- 라이브 아닌 녹화 · 스틸도 같은 영상에서 뽑아 덱과 어긋나지 않음
- 저장된 LLM 판정을 재생 — 데모 중 외부 호출 0

분류 결과를 어디에 남기나

지금은 메모리에만 있음

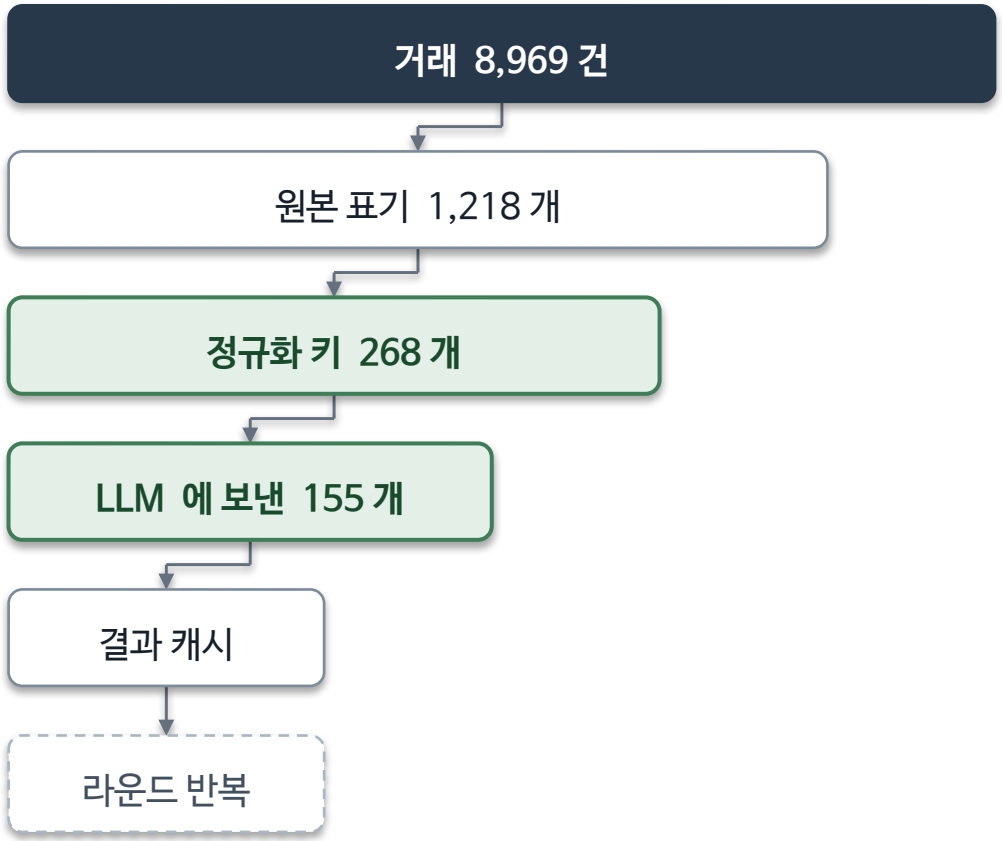
- 분류 결과 8,619 건은 그래프 상태에만 있고 프로세스가 끝나면 사라짐
- 파일로 남는 건 집계 지표 (`eval --json`) 뿐
- **일부러 이렇게 함 :** 시드만 같으면 재현되고 이전 실행의 영향을 안 받음

대신 남기는 것

무엇	어디에	왜
룰 사전 v{n}	`InMemoryStore` namespace `("rules","v{n}")` + `rules/v{n}.yaml`	승격 루프 결과 . 버전 관리 · 롤백 · 감사 추적 필요
검토 큐와 스레드 상태	`InMemorySaver` (checkpointer)	`interrupt()` 로 멈춘 곳에서 이어서 진행 (검토 화면에서 실제 사용)
거래별 분류 결과	남기지 않는다	사전이 원본이고 분류 결과는 파생물 . 사전이 바뀌면 저장한 결과는 바로 낡음

- 화면 (Streamlit) 도 결과를 세션 안에서만 보관
- 라운드마다 바뀌는 값은 캐시하지 않음
- 남은 과제 : 운영에서 매번 전량 재분류 (약 33 분) 는 무리 → 사전 버전 기준 결과 캐시 검토
- 덤으로 거래 데이터가 파일에 안 남음 (규제 측면에서도 유리)

기대 효과



합성 실측 · 고객 30 명 × 6 개월 · 시드 42

전처리 전 · 거래 대비 13.6%

전처리 후 · 실제 LLM 전송 단위 · 거래 대비 3.0%

룰 5 단계가 head 를 덮고 남은 것 · 하이브리드 (C) 실측 · 거래 대비 1.7%

같은 키는 다시 보내지 않음 (히트는 승격 루프 2 회차부터)

사전 성숙 → 외부 호출 0 에 수렴 → 온프레미스 / sLLM 이전

- 비용 : 외부로 나가는 건 전체 거래의 1.7%
- 운영 : 사전 관리가 상시 수작업 → 승격 결과 검토로
- 실무 도입 : 외부로 나가는 건 **가맹점명 문자열뿐** (미들웨어로 강제)

개발 과정 : 1 주차부터 오늘 (6 주차) 까지

1~2 주차 [완료]

주제 확정 · 기획 · 데이터 기반
블루프린트 · 택소노미 · 스키마 · 제공기관 프로파일 · 생성기 · 정답셋

3 주차 [완료]

결정론 파이프라인 + 측정 (중간 발표 · 9/5)
ingest · preprocess (표기 1,218 → 키 268) · rule_match 5 단계 · 평가 하네스
발표한 것 : " 룰만으로는 거래 93.0% · 키 42.0%"

4 주차 [완료]

LangChain 1.0 Agent + 가드레일 + 메모리 + 승격 루프 + 2 부 탐지
create_agent + ToolStrategy · 가드레일 4 개 · (B) (C) 실측 · 실호출 5 라운드 승격 53 건
발표한 것 : 가설 기각 · 승격 루프에 고정점 없음 · 2 부 전이

5 주차 [완료]

사람 교정 (HITL) + 화면 + MCP · 여기서 발표 가능한 완성형
interrupt · human_review → 승격 · S1 교정 세 줄 → 거래 122 건 즉시 반영
S2 는 그 자리에서 다시 잰다 (API 0) · 도구를 MCP stdio 서버로 프로세스 밖에

6 주차 [오늘]

최종 발표 10/17 (10 분 + Q&A · 데모 필수)
승격 루프에 고정점이 없음 , 멈추는 건 사람 판단 · 데모는 화면 녹화로

컷 순서

미리 정해 둔 컷 순서 : 모델 2 단 escalation → S3 거래 탐색기 → 이상거래 후순위 → 골드셋 축소
일정이 안 밀려서 안 씀 . S1 · S2 · S4 · 탐지 3 종 · 룰 5 단계 · 전이 실험 전부 들어감

부록 A. 주제를 어떻게 골랐나

후보 2 안 → 게이트 5 개



G2 는 신용정보법상 목적 외 이용 금지

통합자산 재무진단

○ 된다

× 실데이터 없이 성립 안 함

× 프롬프트 한 번으로도 됨

× 수강생이 가장 많이 고른다

× 좋았는지는 정성 평가

↓

기각
적합성 원칙 · 설명의무가 따라온다

TxLens 거래내역 정규화 · 분류

○ 된다

○ 표기 변형이라 합성으로 재현

○ 분기와 루프가 생긴다

○ 중복 가능성 낮음

○ 정확도 · 미분류율 · 호출수

↓

선정 (D-003)
직후 D-005 로 실데이터 미사용 확정

- 기준을 주제보다 먼저 정했다 . 순서를 바꾸면 이미 마음에 든 주제에 유리한 기준을 나중에 만들게 된다 .
- 재무진단을 접은 결정적 이유는 컴플라이언스다 . 금융상품 추천이 들어가면 적합성 원칙과 설명의무가 따라온다 . 6 주 프로토타입이 감당할 범위가 아니다 .
- 주제를 정하면서 제약을 하나 걸었다 (D-005). 실제 고객 데이터는 한 건도 쓰지 않고 , 발표 자료에도 실데이터나 실화면 캡처를 넣지 않는다 . 시드를 고정하니 재현이 되고 , 생성기가 정답을 알고 있으니 라벨링 노동이 사라졌다 .

부록 B. 설계 결정 18 건

D-	결정	이유	버린 대안
003	주제 = 정규화 · 분류 + 탐지	실무 병목이고 수치화된다	통합자산 재무진단
004	룰 /LLM 하이브리드 + 사전 승격	롱테일이라 역할 분담이 필연	LLM 전량 투입
005	실데이터 미사용 , 합성 전용	신용정보법 목적 외 이용 금지	익명화 실데이터
006	Streamlit 을 발표 매체로	발표 수단을 컷 후보에 둘 수 없다	CLI 데모
010	탐지를 동등한 2 부로	분류의 값을 탐지 숫자로 보인다	탐지를 부가 기능으로
012	제공기관별 편차 모델링	균일 랜덤은 군집 구조를 지운다	regno 를 주 키로
016	6 주 커리큘럼 프레임에 일정을 맞춘다	중간 5 분 / 최종 10 분 · 데모 필수	2~3 주 자체 일정
017	LangChain 1.0 표준 구성	배운 기술로 만들었음이 코드로 보여야 한다	레거시 체인 · 수동 파싱
018	제출 노트북 = 실행 프로그램	설계와 구현이 일치해야 한다	노트북에 로직 복사
024	룰 5 단계 + 단계 0(PG)	확실한 신호가 애매한 신호에 덮이지 않는다	`token_set_ratio` (부분집합이 100 점)
025	A/B/C 가 같은 그래프를 돈다	분류기만 바꿔야 비교가 성립한다	베이스라인마다 별도 스크립트
026	**실측이 가정 3 건을 기각했다**	재보고 대신 가정을 고쳤다	유리한 해석으로 넘어가기
031	노드는 한 문장으로 끝나는 단위	한 노드가 두 일을 하면 경계가 설명이 안 된다	가짜 노드로 그림 늘리기
033	**자동 승격은 새 브랜드를 만들지 않는다**	사전이 판정의 검증자다 . 대표명 오류가 2 부로 전이된다	문턱만 낮춰 자동 승격 늘리기
037	HITL 은 **기본으로 꺼 둔다**	사람 없는 곳에서 멈추면 지표가 낮은 게 아니라 없어진다	항상 켜 두고 자동 승인
040	**화면은 결과가 아니라 과정을 보여준다**	8 노드가 스피너 뒤에 있으면 설계는 말로만 남는다	`invoke` 하나 + 스피너
041	**화면의 수는 그 자리에서 다시 잰다**	적어 둔 수는 사전을 고친 날 화면만 낡는다	슬라이드 수치를 화면에 옮겨 적기
052	도구를 **MCP 서버로 내보낸다**	사람과 에이전트가 같은 질문을 던진다 . 기본 경로는 그대로 (수치 불변)	사람용 도구를 따로 구현

● 나머지를 결정하는 건 D-004 다 . LLM 을 런타임 분류기로 전량 투입하지 않는다 . 근거는 부록 B-2.

부록 B-2. 설계 결정 — 왜 그렇게 정했나

● **D-026 은 이 판에서 새로 남긴 결정이고 셋이 다 내 예상과 반대였다.** ① 세탁소는 택소노미에 자리가 없어 모델의 "모른다"가 정답이었다 → 골드셋 라벨을 고쳤다. ② 택소노미 검색 도구를 막으면 싸질 줄 알았는데 정확도가 떨어졌다 → 되돌렸다. ③ (B) LLM 만이 (A) 룰만보다 정확할 줄 알았는데 11%p 낮았다 → 하이브리드의 논거가 오히려 강해졌다 (S12).

● D-012 를 균일 랜덤으로 하면 평가가 틀린다. 기관별 군집을 없애면 룰이 실제보다 불리해지고, 결과적으로 내 하이브리드가 실제보다 좋아 보인다. 내 주장에 유리한 데이터가 만들어지는 걸 막았다.

● **D-040·D-041 은 화면을 붙이면서 나온 결정이다.** 배점이 Agent 설계 구체성 35 점인데 8 노드가 `invoke` 하나 뒤에 숨어 있었다. 지표 화면도 마찬가지로, 슬라이드 수치를 옮겨 적으면 사전을 고친 날 화면만 옛 수를 말한다 — 그래서 되살릴 수 있는 줄은 화면이 그 자리에서 다시 잰다 (7 초 · API 0).

● D-017 은 이번 개정에서 새로 남겼다. 레거시 체인 (`LLMChain·AgentExecutor`) 과 수동 JSON 파싱은 쓰지 않는다. 무엇을 안 쓰는지 적어 두면 스택 선택의 범위가 분명해진다.

부록 C. 사전 승격 루프



측정 지표 (셋 다 내려가야 한다): 라운드별 LLM 호출 수 ↓ HITL 검토 큐 크기 ↓ 미분류율 ↓

설계

● 사전은 `InMemoryStore` 에 namespace (`"rules", "v{n}"`) 로 올려두고, 승격은 새 namespace 를 쓰는 방식으로 버전을 만든다. 파일 (`rules/v{n}.yaml`) 로도 떨어뜨려 롤백과 감사 추적을 남긴다. v2 의 머리말에 승격 조건 · 판정 모델 · 승격 목록 (표기 → 대표명 · 덮는 거래 수) 을 그대로 적는다.

● 검토 큐를 덮는 거래 건수 × (1 - confidence) 로 정렬한다. 한 건을 고치면 수백 건이 따라 고쳐진다.

● S1 화면이 교정 직후 "사전 v{n+1} 로 올라가서 거래 N 건에 반영됐다" 를 표시한다 — 붙였고, 실측이 세 줄 교정에 122 건이다. 승격 루프가 없으면 이 화면을 만들 수가 없다.

● **폭주를 막는 장치가 층마다 다르다.** Agent 안 (도구↔모델 왕복) 은 `ToolCallLimit·ModelCallLimit`, 그래프는 사이클을 만들지 않는 것 자체가 장치이고, 라운드는 고정점 + 상한인데 실측에서는 상한이 걸렸다 — 수렴과 중단을 같은 말로 보고하지 않는다 (D-056).

부록 C-2. 승격 루프 실행 결과 (D-033 · D-056)

라운드	사전 키	LLM 표기	모델 호출	거래 L2	키 L2	자동 승격	멈춘 이유
1 · v1	65	155	572	98.0%	87.9%	35 건	상한
2 · v2	100	119	438	98.4%	90.5%	8 건	상한
3 · v3	108	111	394	98.1%	91.7%	1 건	상한
4 · v4	115	104	377	98.0%	90.2%	6 건	상한
5 · v5	121	**98**	**351**	97.7%	90.2%	3 건	상한

● **멈춘 이유가 곧 결과다 . 다섯 라운드 전부 상한이다 — 고정점이 아니다 .** 올릴 후보가 매 라운드 남았다 . 처음 보고한 " 라운드 2 고정점 " 은 **저장된 판정을 재생한** 결과였고 , 재생은 판정을 표기 이름으로 들고 있어 **다음 라운드가 커진 사전을 조회하지 못한다** (D-056).

● **승격이 틀려서 멈춘 게 아니다 .** v4·v5 가 더한 13 건은 정답셋 대조 **13/13 맞음**이고 `alias` 단계는 다섯 라운드 전부 **L2 100%** 다 . 오타가 롱테일이라 **고치는 만큼 새로 나온다** — `GS2·GS5·GS52·GSS25` 가 라운드마다 하나씩 올라왔다 .

● **그래서 확인된 값은 호출 감소다 .** 표기 155 → 98(-36.8%) · 호출 572 → 351(-38.6%). **끝나는 자리는 고정점이 아니라 " 남은 것은 사람 몫 " 이라는 판단**이고 , 그 크기가 검토 큐 45 다 .

● **거래 L2 하락 (98.4 → 97.7) 은 승격이 아니라 LLM 분산이다 .** alias 는 100% 고 흔들린 것은 `llm` 단계 (84.9 → 77.2%) 다 . 같은 사전 v3 을 두 번 실호출하니 판정 106 개 중 같은 것이 **32 개** , 대표명이 29 개에서 달랐고 승격이 **1 건 ↔ 7 건**으로 벌어졌다 — **승격 수는 수렴의 증거가 아니다** .

● **문턱은 정답셋으로 골랐다 .** 근거별 정밀도가 `brand_known` 95.9% · `keyword` 78.7% · `guess` 9.1% — 근거가 신뢰도보다 강한 신호다 . 라운드 1 승격 35 건은 키 단위 100% · 거래 단위 97.5%(39/40) 이고 유일한 오답은 D-022 가 예고한 쿠팡 / 쿠팡와우다 — 그것이 라운드 3 에서 **실제로 사전에 굳었다** (HITL 의 표적 · D-035).

● 룰만 (A) 구성도 v2 사전에서 키 L2 42.0 → **54.9%** 로 오른다 . 승격의 값은 LLM 이 없는 경로에도 남는다는 뜻이다 .

● **판정 실패가 라운드마다 2~5 건** 난다 — 재생은 실패할 수 없어 드러나지 않던 현상이다 .

부록 D. 분류와 탐지의 연결

원본 표기

NAVERPAY* 스타벅스 · 스타벅스 강남역점 · ㈜에스씨케이컴퍼니

1 부 정규화 · 분류

merchant_canonical = " 스타벅스 " · category = 카페 / 간식

구독 탐지

같은 canonical 로 묶여야
" 주기 3 회 " 가 채워진다

이상지출 탐지

카테고리가 맞아야
기준선 (직전 3 개월) 이
맞게 잡힌다

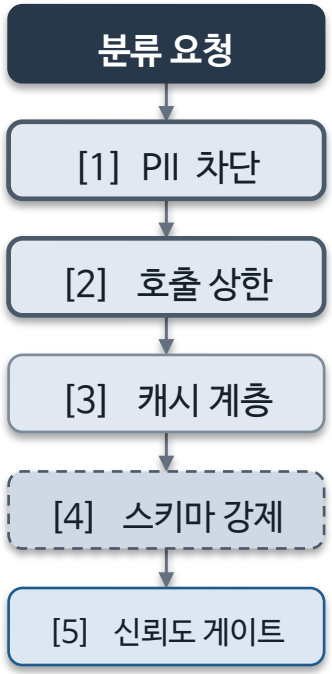
이상거래 탐지

같은 가맹점 판정이 맞아야
중복결제가 보인다

정규화가 안 되면 → 서로 다른 3 개 가맹점 → 3 회를 못 채움 → 구독을 놓친다
카테고리가 틀리면 → 기준선이 잘못 잡힘 → 이상지출 오탐이 늘어난다

" 가맹점명을 못 묶으면 구독도 못 찾는다 " → 전이 실험 (부록 F) 으로 확인한다

부록 E. 규제 제약과 가드레일



고유 가맹점명 문자열 하나

PIIMiddleware(strategy="block") — 고객식별정보 · 카드번호 · 계좌번호 패턴이면 모델에 닿기 전에 차단
민감 업종 차단 목록도 custom detector 로 같은 자리에 붙는다 (미결)

ToolCallLimitMiddleware · ModelCallLimitMiddleware — 라운드당 호출 상한
비용 상한을 문서가 아니라 코드로 고정한다

wrap_model_call — 같은 문자열은 모델을 부르지 않는다 (캐시 히트)

모델 호출 → ToolStrategy(LLMVerdict) — Enum 밖 카테고리는 검증에서 막힌다
실패 시 재시도 , 억지 통과 없음

confidence_gate → 낮으면 사람에게 (interrupt)

	흔한 LLM 적용	TxLens
전송 단위	거래 건 (고객 · 금액 · 일시가 붙는다)	고유 가맹점명 문자열
개인 식별	가능	문자열만으로는 불가
차단 지점	프롬프트 규칙 (지켜지는지 알 수 없다)	미들웨어 (코드로 강제 , 테스트 가능)
재전송	매번	캐시 히트로 0
장기 경로	외부 의존이 고정된다	사전 성숙 → 호출 0 → 온프레미스

부록 F. 베이스라인 실측 (전문)

베이스라인 3종 (1부)

구성	거래 L1	거래 L2	키 L2	확정 커버리지	미매칭	PG 마스킹	LLM 키	모델 호출
(A) 룰만	93.0%	93.0%	42.0%	84.3%	7.7%	8.0%	0	0
(B) LLM 만	89.2%	85.6%	66.7%	86.8%	5.1%	8.0%	257	1,026
(C) 하이브리드	99.0%	98.9%	86.7%	90.3%	1.7%	8.0%	155	580

단계별 기여 — 확정 단계는 어느 구성에서도 100% 다

단계	(A) 건수 / L2	(B) 건수 / L2	(C) 건수 / L2
`pg_masked`	691 / 100.0%	691 / 100.0%	691 / 100.0%
`exact`	5,474 / 100.0%	—	5,474 / 100.0%
`alias`	1,746 / 100.0%	—	1,746 / 100.0%
`regex · fuzzy`	7 · 35 / 100.0%	—	7 · 35 / 100.0%
`llm`	—	7,485 / **88.4%**	521 / **97.1%**
`unmatched`	666 / 9.9%	443 / 14.9%	145 / 45.5%

● `llm` 단계만 88.4% 와 97.1% 로 갈린다. 같은 모델, 같은 프롬프트, 같은 도구이고 차이는 앞에서 룰이 head 를 덮었는지 하나뿐이다. 하이브리드에서 정확도와 비용이 함께 개선되는 이유가 여기 있다.

● `unmatched` 가 맞은 몫은 룰의 성과로 볼 수 없다. 정답이 미분류 인 거래를 미분류로 둔 것이다 (택소노미에 자리가 없는 업종. D-026①). 지표 출력에 이 설명을 함께 찍어 오독을 막았다.

부록 F-2. 베이스라인 실측 — 무엇을 틀렸나 · 기관별

혼동 상위 — (B) 가 무엇을 어떻게 틀렸나

정답 → 예측	(B) 2026-08-31	(C) 이전 실행
`shopping → food`	**500**	4
`cafe → unclassified`	243	34
`culture → unclassified`	60	5
`health → unclassified`	32	—
`health → shopping`	**0**	0

● **1 위 오분류는 재현되고 꼬리는 재현되지 않는다.** 같은 구성을 다시 돌렸을 때 `shopping → food` 는 497 → 500 으로 그대로였지만 `health → shopping` 은 106 → **0**, `culture → unclassified` 는 111 → 60 으로 움직였다. 주장의 근거는 1 위이고, 꼬리 숫자를 근거로 쓰면 안 된다 (D-048).

● (B) 는 2026-08-31 재실측이고 (A)·(C) 는 이전 실행이다. (A) 는 룰만 쓰므로 결정론이지만 **(C) 는 아니다** — 같은 표에 두 실행이 섞여 있다는 뜻이고, 그래서 B↔C 차이는 구성 차이와 실행 간 변동이 함께 들어간 값이다. (A)↔(B) 비교에는 이 문제가 없다.

제공기관 축 분해 — 프로파일 (S03) 이 지표에 그대로 나타났다 (D-012)

기관	건수	(A) L2 / 커버리지	(C) L2 / 커버리지	PG
CARD_A	3,033	95.7% / 94.0%	99.7% / 98.1%	1.0%
CARD_B	2,807	91.3% / 82.7%	98.1% / 89.6%	7.8%
CARD_C	752	93.4% / 67.2%	99.3% / 73.3%	26.2%
CARD_E	917	89.7% / 76.0%	98.1% / 85.2%	13.2%
EFIN_D	1,110	92.8% / 79.9%	99.0% / 86.4%	11.3%

부록 F-3. 분류 → 탐지 전이 실험 (전문)

분류 단계	구독 P/R	급증 P/R	고액 P/R	이상거래 P/R	알림 / 월
정규화만	95.3 / 56.2	—	—	96.4 / 79.4	0.2
(A) 룰만	94.7 / 74.0	35.7 / 34.5	89.7 / 86.7	97.1 / 100.0	0.5
(C) 하이브리드	94.7 / 74.0	36.4 / 41.4	90.3 / 93.3	97.1 / 100.0	0.6
정답 분류 (상한)	95.7 / 90.4	37.1 / 44.8	90.6 / 96.7	97.1 / 100.0	0.6

● 도구 : `pytest` 로 결정론 단계 회귀 고정 (384 건 — 2 부 ·HITL· 화면 ·MCP· 제출 노트북 포함). 수치는 하네스의 `RunMetrics` 로 쟀다 — LangSmith 는 코드에 붙이지 않았다 (환경변수만 켜면 같은 그래프가 그대로 트레이싱된다).

부록 F-4. 비용 · 재현성 — 같은 구성도 수가 움직인다

● 전량 실행 모델 호출 1,606 회 (B 1,026 + C 580), **승격 루프 5 라운드 2,549 회**, 사전 점검 · 재실측 포함 약 5,800 회, **합계 약 3 달러** (개산). 전량 1 회가 약 31 분, 승격 라운드 하나가 약 7 분이다.

● `gpt-4o-mini` **단일 모델**이고 2 단 escalation(mini → 4o) 은 측정하지 않았다 — 컷 후보 1 (D-010). 모델을 바꾸면 수치가 바뀐다는 점을 한계로 명시한다.

● **같은 구성을 다시 돌려도 수가 움직인다.** (B) 를 2026-08-31 에 재실측했더니 거래 L2 가 82.0 → 85.6% (**±3.6pp**), 호출이 1,059 → 1,026 이었다. 1 위 오분류는 재현되고 꼬리는 재현되지 않는다 (부록 F-2). 그래서 표에 실행 일자를 적고, 주장을 꼬리 숫자에 걸지 않는다 (D-048).

● **그 분산이 정확도만 흔드는 게 아니다.** 같은 사전 v3 을 두 번 실호출하니 판정 106 개 중 같은 것이 32 개뿐이고 대표명이 29 개에서 달랐다. 그 결과 **승격이 1 건 ↔ 7 건**으로 벌어졌다 — 루프의 **종료 조건까지** 흔든다는 뜻이라 승격 수를 수렴의 증거로 쓰지 않는다 (D-056).

● (C) 에서 판정 실패 6 건, "모른다" 18 개, 검토 큐 75 개. 실패를 미분류 에 합치지 않고 따로 센다. 5 주차에 이 75 개가 사람 검토로 넘어간다.

● **승격의 호출 감소는 이제 실측이다.** 라운드 1~5 를 실호출로 돌려 표기 155 → 98, 호출 572 → 351 (−38.6%) 을 쟀다 — 합계 2,549 호출 · $1.3 개산 (부록 C-2). 재생으로 보고한 "라운드 2 고정점" 은 틀렸고, 다섯 라운드 전부 상한에서 멈췄다 (D-056).

● **그 과정에서 플래그 하나로 417 호출을 날렸다.** 상한에서 멈추면 그 라운드의 승격은 사전 파일로 나가지 않는다 (그 사전으로 잰 지표가 없으므로 의도된 동작). `--rounds 1` 로 돌려 승격 7 건이 조용히 버려졌다 — 표가 그 사실과 되찾는 방법을 찍도록 고쳤다.

● 전량 실행에서 캐시 히트 0 — 라운드 1 회라 재전송이 없다. 라운드를 이어 돌리면 저장된 판정 재생으로 호출 0 이 되지만, **재생으로 종료 조건은 판정하지 않는다** (D-056).

부록 G. 1~4 주차 실측 결과

항목	실측값	의미
거래	8,969 건	고객 30 명 × 6 개월 (2026-03-01~08-31), 시드 42
원본 표기	1,218 개	전처리 **전**. 거래 대비 13.6%
정규화 키	268 개	전처리 **후**. 실제 LLM 전송 단위. 거래 대비 3.0%
분류 대상	8,619 건 / 키 264 개	취소 상계 175 + 취소 거래 175 제외
PG 마스킹 정답	691 건 = 8.0%	원리적 하한. **세 구성에서 모두 같다** → 정확도 상한 92%
룰 사전 v1	head 29 개 · 5 단계 + 단계 0(PG)	(A) 거래 L2 93.0% / 키 L2 42.0% (D-024)
베이스라인 (B) LLM 만	거래 L2 85.6% · 호출 1,026	**(A) 보다 7.4%p 낮다**. 가설 기각 (D-026③) · 2026-08-31 재실측 (D-048)
베이스라인 (C) 하이브리드	거래 L2 98.9% · 호출 580	미매칭 7.7 → 1.7%. LLM 단계 97.1%
승격 루프	실호출 5 라운드 · **고정점 없음** (전부 상한)	LLM 표기 155 → **98**(36.8% 감소) · 호출 572 → **351** · 사전 65 → 121 키 (D-056)
자동 승격 판정	5 라운드 53 건	`alias` 단계 L2 **100%** · v4·v5 분 13 건은 정답셋 **13/13**. 별칭만 올린다 — 새 브랜드는 사람 몫 (D-033)
룰 사전 v2	head 29 개 + 자동 별칭 34 개	(A) 룰만 구성도 키 L2 42.0 → **54.9%** 로 오른다
테스트	359 passed	ruff clean. HITL 28 건 · S1 화면 16 건 · S2 화면 16 건

- 정답셋을 생성 시점에 함께 기록하도록 만들어서 수동 골드셋 라벨링이 필요 없어졌다.
- 원본 표기 1,218 과 정규화 키 268 은 다른 수다. 섞어 쓰면 감소폭이 틀어진다. 앞 판에서 전송 단위 감소를 86.4% 로 적었는데, 실제 전송 단위는 키라서 97.0% 다 (룰까지 거치면 98.3%).

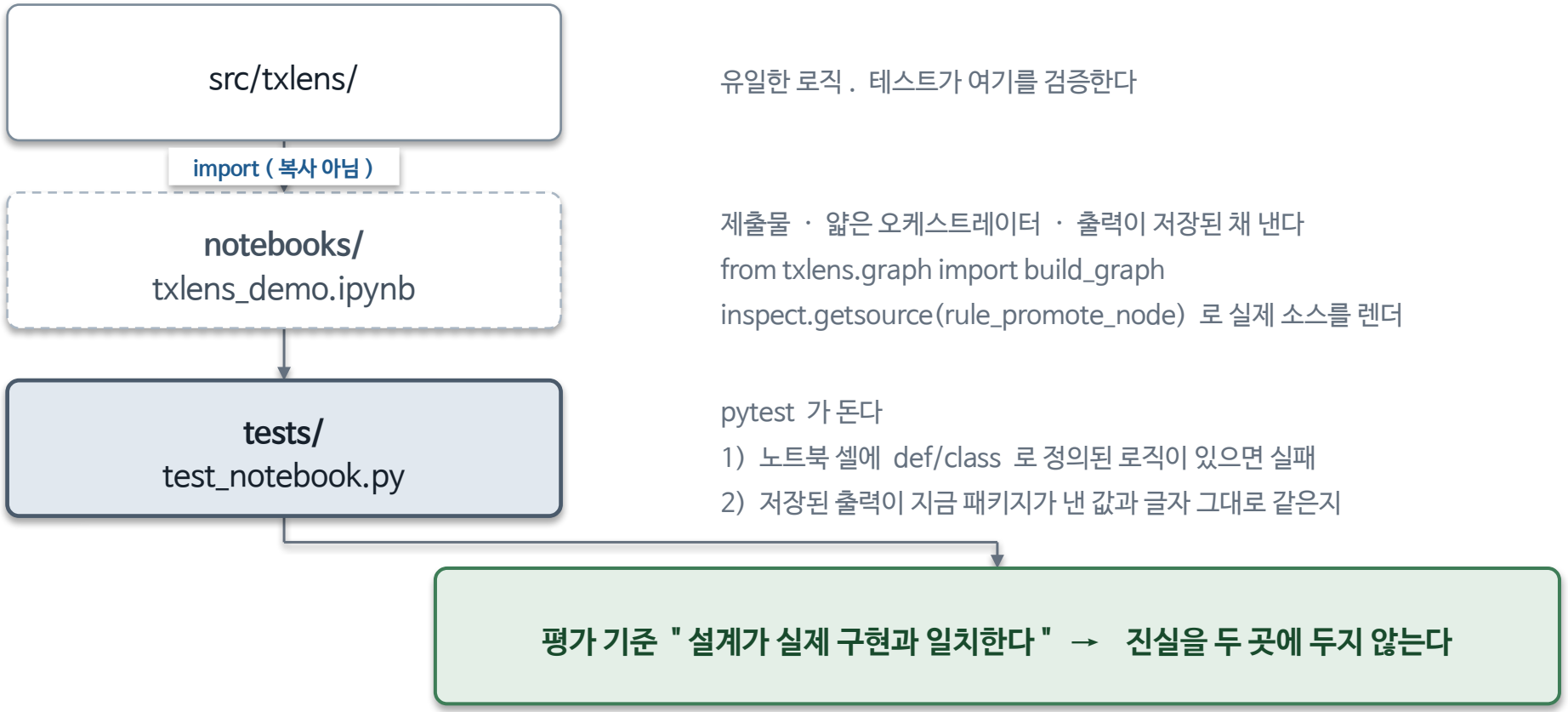
부록 G-2. 2 부 실측 결과

항목	실측값	의미
구독 (정답)	90 건 (탐지가능 73 건)	연 구독 17 건은 6 개월 창에서 원리적으로 못 잡는다
구독 탐지	P 94.7 / R 74.0 (사전 v1)	상한 (정답 분류) 은 R **90.4**. 격차는 오귀속이다 (S14)
이상지출 (정답)	60 건	급증 P 35.7 · R 34.5 / 단발 고액 P 89.7 · R 86.7
이상거래 (정답)	34 건	P 97.1 · R **100.0** — 분류에 가장 덜 의존하는 탐지기다
오탐 함정	60 건	구독처럼 보이는 비구독 30 + 정상 연속결제 30 — **통과 0 건**
고객당 월 알림	0.5 건	알림은 읽히지 않으면 없는 것과 같다 (blueprint §3.6)
전이 (구독 재현율)	56.2 → 74.0 → 90.4	정규화만 → 룰만 → 정답 분류 . 표지의 주장 (S14)
프로파일 준수	regno 96/95 · 48/50 · 22/20%	제공기관별 회신율이 설계값대로 나왔다 (D-012)

● 정확도 목표를 절대치로 잡지 않았다 . 미분류 8.0% 가 하한이므로 **A→C 개선폭** (93.0 → 98.9%) 과 전이 실험의 P/R 차이로 잡는다 . 후자도 이제 실측이다 .

● 탐지 3 종은 LLM 을 호출하지 않는다 . 같은 입력이면 같은 출력이고 , 근거 문장이 규칙을 그대로 옮긴 문장이라 고객에게 설명할 수 있다 — 금융 앱에서 설명 가능성이 정확도보다 중요할 때가 있다 .

부록 H. 최종 제출물 — 노트북과 실행 프로그램의 일치



부록 I. 한계와 위험

한계 / 위험	대응
합성 데이터라 실분포와 차이가 있다	한계로 명시한다. 기관별 편차는 실무 확인으로 모델링했고 오탐 함정도 넣었다
PG 마스킹은 못 푼다	억지로 추측하지 않고 미분류 + `pg_masked` 로 둔다. 상한 92% 를 먼저 공개한다
5 주차 과부하	컷 순서를 미리 정했다. 4 주차 종료 시점이 이미 발표 가능선이다
라이브 데모 실패 (6 주차 필수 항목)	**라이브로 하지 않는다** — 화면을 녹화해 두고 질문 시간에 틀며, 본편에는 스틸 한 장 (S14-2) 이 들어간다. replay 모드라 다시 뜰 수 있다. 남는 위험은 재생 환경 하나이고 H.264 · yuv420p · 무음은 녹화 스크립트가 검사한다 (D-053 · D-059)
`InMemoryStore` 는 프로세스가 죽으면 사라진다	사전은 파일로도 떨어뜨린다. 운영 전환 시 DB 백엔드 store 로 교체 (같은 인터페이스)
`gpt-4o-mini` 단일 모델로만 측정했다	2 단 escalation 은 컷 후보 1 이다. 모델을 바꾸면 S12 의 수치가 바뀐다는 점을 한계로 명시한다
캐시 히트가 0 이었다	라운드 1 회 실행이라 재전송이 없어서 당연하다. 캐시의 값은 승격 루프 2 회차부터 측정된다 (부록 C)

부록 I-2. 미결 — 다음 작업

미결	상태
승격 루프가 수렴하지 않는다	실호출 5 라운드가 전부 상한에서 멈췄다. 승격은 정답셋 대조 전부 맞았고 호출은 38.6% 줄었지만, 오타가 롱테일이라 후보가 계속 나온다. **어디서 멈출지는 사람이 정한다** — 자동 종료 조건을 더 두지 않는 것이 결론이다 (D-056)
S2 지표 화면의 (B) 줄	**해결.** (A)·(C)·전이·승격 라운드는 화면이 그 자리에서 다시 재는데 (B) 만 사전 없이 표기 전체를 보내는 구성이라 되살릴 수 없어, 2026-08-31 실호출 스냅샷 (`--json data/gold/baselines-v1-gpt-4o-mini.json`, 1,026 호출) 으로 읽는다 (D-041②). **다만 수가 움직였다** — 거래 L2 82.0 → 85.6%. 슬라이드는 화면이 읽는 이 수를 싣는다 (D-048)
자동 승격이 오답을 사전에 굳힐 위험	대표명을 사전에서 찾을 수 있을 때만 승격한다 — 사전이 검증자다 (D-033). 키 단위 100%· 거래 단위 97.5%, 유일한 오답은 D-022 가 예고한 쿠팡 / 쿠팡와우다
(C) 에서 판정 실패 6 건 · "모른다" 18 개	미분류 에 합치지 않고 따로 센다. 검토 큐 75 개가 5 주차에 사람 검토로 넘어간다
급증 탐지의 정답셋이 성기다	※ 미결. 생성기가 심은 급증 중 여럿이 카테고리 월 합계를 움직이지 않는다 (싼 브랜드의 건수만 늘려서) — 정밀도 37% 는 하한이다. 문턱을 정답셋에 맞춰 구부리지 않고 제품 관점 (1.5 배 + 10 만원) 으로 정했다. 생성기 수정은 결국 하지 않았다 — 지금 데이터를 다시 만들면 이 문서의 1 부 실측이 전부 바뀐다 (부록 F-3)
구독 탐지의 남은 손실이 커버리지가 아니다	74.0 → 90.4 의 격차는 사전이 **잘못 묶은 것**이다 (쿠팡와우→쿠팡). 사람 교정의 표적을 여기로 잡았다 (S14)
가맹점명 자체의 추론 위험	※ 미결. 특정 병원·종교시설 이름은 그 자체로 민감정보 추론 단서가 될 수 있다. `PIIMiddleware` 의 custom detector 로 민감 업종을 차단하는 방안을 검토 중이나, 업종 목록의 기준은 판단이 필요하다
주차 간격이 균일하지 않다 (중간 9/5 → 최종 10/17, 달력 6 주)	토요일 하루 수업이고 사이에 비수업 주가 있다. 4·5 주차 강의일은 아직 확인하지 못했다. 달력 여유가 커리큘럼 주차보다 길어 버퍼로 썼다 — 안 밀려서 컷 순서는 쓰지 않았다

부록 J. Q&A — 설계 · 기술

질문	답
룰만 잘 만들면 되지 않나	그게 (A) 입니다. 거래 기준 93.0% 인데 정규화 키 기준은 42.0% 입니다. 건수로 보면 되는 것처럼 보이고 종류로 보면 절반이 안 됩니다. 그게 롱테일이고, 신규 브랜드가 계속 들어와 유지보수가 끝나지 않습니다
LLM 에 전부 넣으면 더 정확하지 않나	측정해 봤고 더 부정확했습니다. (B) 85.6% 대 (A) 93.0% 입니다. 커버리지는 (B) 가 높은데 정확도가 낮습니다. 답을 더 많이 내지만 그만큼 오분류가 늘어납니다. 하이브리드는 98.9% 이고 호출은 (B) 의 절반입니다
왜 (B) 가 (A) 보다 낮은가	사전이 없으면 법인명을 풀지 못합니다. 에스씨케이컴퍼니 200 건이 그렇습니다. 편의점 · 마트도 식비로 500 건 보냈습니다. 오분류가 계통적으로 몰립니다. 룰이 앞에서 head 를 덮으면 같은 모델의 LLM 단계 정확도가 88.4 에서 97.1 로 올라갑니다
(C) 의 unmatched 가 45.5% 나 맞았다는 건 뭔가	정답이 미분류 인 거래를 미분류로 둔 것이 맞은 겁니다. 룰의 성과가 아니라 택소노미에 자리가 없는 업종입니다. 성과로 읽히지 않게 지표 출력에 문장을 붙여 뒀습니다 (부록 F)
그럼 사전을 계속 키우면 LLM 이 필요 없지 않나	그게 목표입니다. 승격 루프가 그 일을 하고, 외부 호출이 0 에 수렴하면 온프레미스로 옮깁니다. 다만 신규 브랜드는 계속 들어오므로 tail 담당은 남습니다
임베딩이나 파인튜닝은 왜 안 쓰나	학습 데이터에 라벨이 필요하고, 그 라벨을 만드는 게 지금 푸는 문제입니다. 승격 루프가 라벨을 쌓으니 그 다음 단계입니다
PG 마스킹은 어떻게 푸나	못 풉니다. 억지로 추측하면 오분류가 늘어 신뢰를 잃습니다. 미분류 로 두고 상한 92% 를 먼저 공개합니다
FDS 와 뭐가 다른가	FDS 가 아닙니다. 차단이 아니라 확인해 보라는 알림이고 판단은 고객이 합니다

부록 J-2. Q&A — 승격 루프와 폭주 방지

질문	답
무한루프는 어떻게 막나	층마다 다른 장치입니다. Agent 안의 도구↔모델 왕복은 `ToolCallLimit·ModelCallLimit`, 그래프는 사이클을 만들지 않는 것 자체가 장치 (DAG), 승격 라운드는 고정점이 종료 조건이고 라운드 상한은 안전핀입니다. **실측에서는 다섯 라운드 전부 상한이 걸렸고, 그래서 표에 "수렴 아님"이라고 적었습니다** — 안전핀이 실제로 일을 한 경우입니다 (D-056)
승격 루프를 왜 그래프 사이클로 안 만들었나	같은 실행에서 되돌면 LLM 이 잡은 거래가 다음 바퀴에 `exact` 로 재분류돼 단계 분포가, 즉 비교표가 흔들립니다. 그래프는 DAG 로 두고 루프는 라운드 사이 (v1 → 승격 → v2 → 재측정) 에 뒀습니다
자동 승격이 오답을 사전에 굳히면 어떻게 되나	그게 제일 큰 위험이라 자동은 **별칭까지만** 합니다. 대표명을 사전에서 찾을 수 있을 때만 올리므로 사전이 검증자입니다. 다섯 라운드에서 `alias` 단계는 늘 L2 100% 였습니다. 다만 **예고해 둔 쿠팡 / 쿠팡와우가 라운드 3 에서 실제로 굳었습니다** — 그 한 건이 HITL 이 먼저 볼 자리입니다 (D-033·D-056)
문턱 0.95 에서 0 건이었으면 그냥 낮추면 되지 않나	낮출 수는 있는데 **얼마로** 낮출지가 근거 없는 숫자가 됩니다. 그래서 155 개를 전수 판정해 저장하고 문턱별로 정답셋과 대조했습니다. 판정을 저장해 뒀으니 문턱 재실험은 이제 모델 호출 0 회입니다
승격했는데 정확도가 떨어지면	그건 자동 승격이 오답을 굳혔다는 신호이고, 표에 거래 L2 를 함께 찍는 이유입니다. 이번에는 98.9 → 99.0 으로 떨어지지 않았습니다. 떨어졌다면 되돌립니다 — `rules/v{n}.yaml` 이 그래서 파일로 남습니다

부록 J-3. Q&A — Agent 구성과 도구 경계

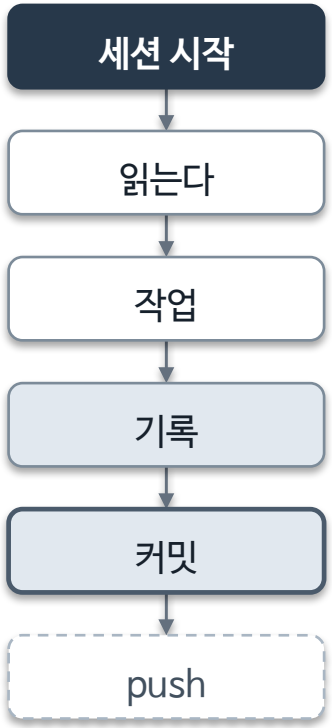
질문	답
파이프라인인데 왜 Agent 인가	파이프라인 골격은 `StateGraph` 로 짜고, Agent 는 `llm_classify` 노드 안에만 둡니다. tail 분류에서 사전 · 택소노미를 조회하는 툴 콜이 실제로 필요합니다
왜 `SqliteSaver` 가 아니고 `InMemorySaver` 인가	데모와 노트북 재현성에는 InMemory 가 맞습니다. 프로세스를 넘겨야 하면 같은 인터페이스로 DB 백엔드로 바꿉니다. 사전은 별도로 파일에 떨어뜨립니다
MCP 를 왜 붙였나 — 억지로 끼운 것 아닌가	이미 어긋나 있던 자리를 골랐습니다. 에이전트 도구가 답하는 질문이 승격 루프에서 사람이 던지는 질문과 같은데, 코드는 `llm_classify` 안에만 있었습니다. 도구를 두 번 만들지 않으려고 `make_tools()` 를 감싸기만 했습니다 (S10 · D-052)
MCP 를 붙였는데 왜 수치가 안 움직이나	기본 분류 경로를 MCP 로 바꾸지 않았습니다. MCP 도구는 async 전용이라 그래프를 async 로 올려야 하고, 전달 방식이 바뀌면 A/B/C 표를 다시 재야 합니다 — 같은 구성을 다시 돌려도 수는 움직입니다 (D-048). 경로는 열어 두고 양쪽 답이 같은지는 테스트가 봅니다

● **부록 J 에서 네 행을 여기로 옮겼다.** 같은 표에 두면 슬라이드가 축소 바닥에 가까워진다 (뒷줄에서 못 읽는다 — D-034). 나누는 기준은 "에이전트를 어떻게 구성했나" 다.

부록 K. Q&A — 실무 · 평가 · 범위

질문	답
실무에 정말 쓸 수 있나	외부로 나가는 건 가맹점명 문자열뿐이고 미들웨어로 강제합니다. 사전이 자라면 외부 호출이 0 에 수렴해 온프레미스로 옮길 수 있습니다 (부록 E)
가맹점명도 개인정보 아닌가	문자열만으로는 특정 개인을 식별할 수 없고, 정규화 키 단위라 거래 맥락과 분리됩니다. 다만 민감 업종명은 추론 단서가 될 수 있어 차단 방안을 검토 중입니다
합성 데이터로 한 실험이 의미가 있나	한계입니다. 다만 이 문제의 본질이 표기 변형이라 합성으로 재현되는 부분이 크고, 기관별 편차는 실무 확인으로 모델링했고 오탐 함정으로 precision 을 재게 했습니다
정확도 목표가 몇 % 인가	절대치로 잡지 않습니다. 미분류 8.0% 가 하한이고, 실측 개선폭이 A 93.0% → C 98.9% 입니다. 그게 탐지로 얼마나 전이되는지도 쟀습니다 — 구독 재현율 56.2 → 90.4 입니다 (S14)
가설이 틀렸는데 그대로 발표하나	그대로 발표합니다. 기각된 방향이 결과적으로 설계를 뒷받침합니다. 유리하게 해석하는 대신 골드셋 라벨과 프롬프트를 고쳤고 결정으로 남겼습니다 (D-026)
1 부를 잘하는 게 왜 중요한가	표로 답합니다. 같은 탐지기를 이름만 정규화한 결과 위에서 돌리면 구독을 56.2% 밖에 못 찾고 이상지출은 아예 0% 입니다. 분류가 완벽하면 90.4% 입니다. 1 부는 목적이 아니라 2 부의 전제입니다 (S14)
다른 팀과 차별점은	사전 승격 루프입니다. 실호출 다섯 라운드에 LLM 이 볼 표기가 155 → 98, 호출이 572 → 351 로 줄었습니다. 그리고 **멈추지 않았습니다** — 오타가 롱테일이라 어디서 끝낼지는 사람이 정합니다 (S13)
사람 교정으로 무엇을 먼저 고치나	커버리지가 아니라 **오귀속**입니다. 구독 재현율 74.0 → 90.4 의 격차가 전부 사전이 잘못 묶은 것 (쿠팡와우→쿠팡) 에서 나옵니다. 검토 큐 정렬에 이걸 반영합니다 (S14)
사람이 얼마나 붙어야 하나	숫자로 답합니다. 자동 승격 53 건 뒤에도 표기 98 개가 남고 검토 큐는 45 개입니다. 큐는 덮는 거래 건수 × `(1 - confidence)` 로 정렬합니다 — 한 건을 고치면 수백 건이 따라 고쳐집니다
6 주에 다 되나	4 주차 종료 시점이 이미 발표 가능선이고 컷 순서를 미리 정했습니다 (S17)
코드는 어떻게 제출하나	GitHub Public Repo URL 과 노트북 둘 다 냅니다. 노트북은 패키지를 import 하는 얇은 층이라 실행 프로그램과 어긋나지 않습니다 (부록 H)

부록 M. 개발 방법론



AI 세션 컨텍스트는 매번 초기화된다

읽는 양을 고정한다 (이력이 길어져도 늘지 않는다)
current-task 전문 → session-log 최근 3 개 → decisions 인덱스 표 → todo 현재 Phase → git log -10

완료한 일 → session-log 최상단 설계 결정 → decisions.md D- 번호 + 근거 상태 → todo / current-task

pre-commit 훅 : scripts/check-docs-budget.sh — 상한 초과 시 커밋 차단
상한을 늘리는 대신 archive/ 로 회전한다

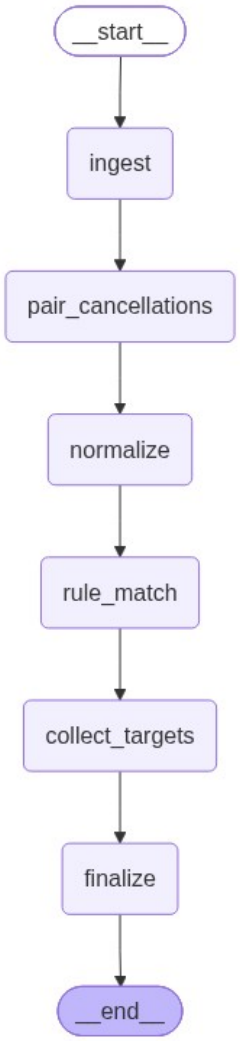
원격에 있는 것만 확실히 안전하다

- 이 프로젝트는 AI 에이전트와 함께 개발한다 . 세션이 매번 초기화되는 문제를 문서로 해결했다 . 결정에 D- 번호를 붙여 근거를 남기니 다음 세션이 같은 논쟁을 반복하지 않는다 .
- 기록 파일은 계속 쌓이는데 세션 시작에 그걸 읽으라고 하면 시작 비용이 이력 길이에 비례한다 . 그래서 읽는 범위를 고정하고 상한을 pre-commit 훅으로 강제했다 (D-008/D-011).
- 발표에서는 부록으로만 둔다 . 주제가 아니라 작업 방식이라서다 .

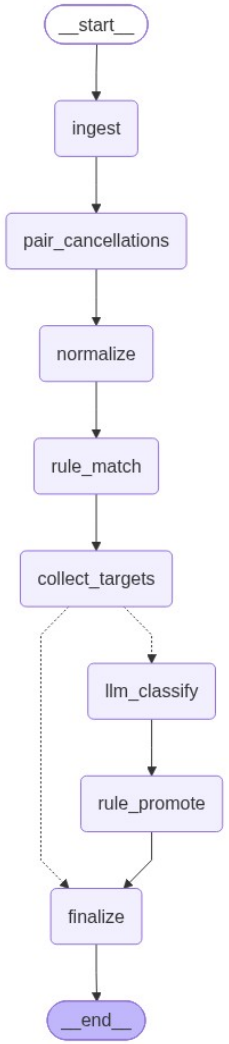
부록 N. 실제 컴파일되는 그래프

- 오른쪽 두 장 : A 에는 `llm_classify` 노드가 없고 , C 는 `collect_targets` 에서 점선 분기가 갈린다
- 손으로 그리지 않았다 . `build_graph().get_graph()` 출력 그대로다 (`python scripts/render-graph.py` — mermaid PNG · ASCII · mermaid 원문 3 종)
- S05 는 설계도 (13 노드), 이 장은 현재 컴파일되는 그래프 (C 8 · A 6) 다 . 그 차이가 5 주차 작업 범위다 — `confidence_gate`·`human_review`·`report`. `rule_promote` 는 이번에 빠져나왔다 (S13). 탐지 3 종은 여기 없는 게 맞다 — 그래프 밖의 순수 함수로 두었다 (D-035)
- A 와 C 의 그래프가 다르다는 것이 D-025② 의 근거다 . `classifier=None` 이면 노드를 만들지 않는다 . 만들어 두고 방문하지 않으면 그림과 실행 경로가 어긋난다
- 코드가 바뀌면 이 그림도 낡는다 . mermaid 원문 (`.mmd`) 을 함께 커밋하고 `python scripts/render-graph.py --check` 로 불일치를 잡는다 (네트워크 불필요)
- PNG 렌더는 mermaid.ink 를 거친다 . 전송되는 것은 노드 이름뿐이고 거래 데이터는 포함되지 않는다 . 발표장 네트워크에 의존하지 않도록 PNG 를 커밋해 둔다 (D-029)

- **`llm_classify` 안의 Agent 그래프**는 `assets/diagrams/langgraph-agent.png` 에 따로 있다 (`build_classifier(...).agent.get_graph()`). 노드 이름이 길어 슬라이드에 넣으면 읽히지 않으므로 S08 에는 같은 내용을 텍스트로 옮겨 두었다 — 원본이 필요하면 이 파일을 연다

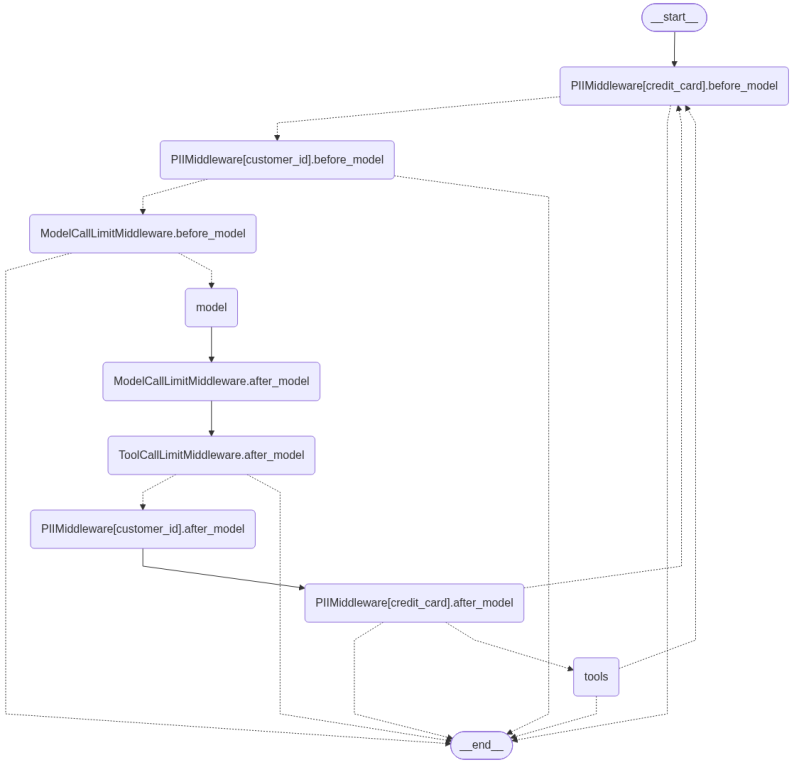


A · 룰만



C · 하이브리드

부록 N-2. llm_classify 안의 Agent 그래프 (실제 렌더)



create_agent 가 만든 그래프 (10 노드)

- `build_classifier(...).agent.get_graph()` 출력이다. 손으로 그리지 않았고 `--check` 가 잠근다
- 미들웨어 6 노드 + `model` + `tools` + 시작 / 끝 = 10 노드. 가드레일마다 `__end__` 로 가는 점선이 있다
- 라벨에 대괄호가 들어가 LangChain 의 `draw_mermaid()` 출력을 mermaid.ink 가 거절한다. 라벨을 따옴표로 감싸는 emitter 를 `scripts/render-graph.py` 에 따로 뒀다 (D-031)

부록 O. 본편 슬라이드의 보충 근거

S06 State · S07 Node/Edge

- `human_corrections` 를 늦게 넣은 이유 — 생산자도 소비자도 없는 필드가 있으면 상태 정의만 보고 HITL 이 이미 붙어 있다고 읽힌다 (D-025)
- `ReviewState` 를 따로 둔 이유 — 그래야 화면이 검토 큐만으로 같은 서브그래프를 돌리고, 화면과 배치가 다른 코드로 갈라지지 않는다 (D-037)
- `Send` 병렬을 안 쓴 이유 — 표기 순서가 고정돼야 캐시 히트 수와 호출 수가 결정론이 되고, 그 수치를 발표 지표로 쓴다. 지금 측정 대상은 지연이 아니다

S08 Agent 내부

- 도구 호출 횟수는 **모델이 정한다** — 그래서 표기 하나에 모델 호출이 2 회 이상 발생한다
- 가드레일은 문서가 아니라 코드다 — 고객식별정보 · 카드번호가 섞이면 모델에 닿기 전에 끊기고, 비용 상한도 `run_limit` 으로 들어가 있다

부록 O-2. 본편 슬라이드의 보충 근거 (실측 장)

S13 승격 루프

- 문턱 0.95 에서 승격이 0 건이었던 원인은 코드가 아니라 **few-shot 이 0.85~0.9 만 보여준 척도**였다 . 155 개를 전수 판정해 저장하고 근거별 정밀도를 다시 쟀다
- 모델은 확신이 없으면 **입력 표기를 그대로 대표명으로** 돌려준다 . 1 부의 실수가 2 부 집계로 전이된다 — 이 프로젝트의 주장이 무너지는 자리다
- 두 문턱이 겹치는 구간 (0.7~0.8) 에서 **사람을 앞에 뒀다** — 뒤집히면 사람이 화면에서 고친 답이 조용히 버려지고 화면은 " 반영됨 " 이라 적는다 (D-037)
- 표기 155 → 115 는 **한 라운드 + 사람 교정 세 줄**의 수다 . 자동만 다섯 라운드 돌리면 98 이다

S14 2 부 · S15 결과 보존 · S16 효과

- 구독 재현율 56.2% 의 정체 — 넷플릭스가 NETFLIX · 넷플릭스 · NAVERPAY* 넷플릭스 로 쪼개져 " 최소 3 회 " 조건을 못 채운다
- 운영 캐시의 후보는 **결과 테이블에 사전 버전을 키로** 넣어 무효화하는 방식이다 . 적용은 6 주차
- 거래 데이터가 파일에 남지 않는 것은 규제 관점에서 유리하지만 **목표는 재현성이었다**
- 승격 루프가 바꾸는 담당자의 일 — 사전을 전부 채우는 대신 **거래 건수가 많은 항목부터** 본다